Object Oriented Programming I

September 18, 2024

Content

Operator Overloading

Overloading Binary Operators

Overloading the << and >> Operators

Overloading Unary Operators

Overloading the Unary Prefix and Postfix ++ and −− Operators

Introduction

- You can use operators with your own user-defined types.
- Although C++ does not allow new operators to be created,it does allow most existing operators to be overloaded.
- So that, when they're used with objects, they have meaning appropriate to those objects.
- Operator overloading is not automatic which means that you must write operator-overloading functions to perform the desired operations.
- An operator is overloaded by writing a **non-static member** function definition or **non-member function** definition as you normally would, except that the function name starts with the keyword operator followed by the symbol for the operator being overloaded.
- For example, the function name operator+ would be used to overload the addition operator (+) for use with objects of a particular class.
- When operators are overloaded as member functions, they must be non-static, because they must be called on an object of the class and operate on that object.

Rules and Restrictions on Operator Overloading

- **The precedence of an operator cannot be changed by overloading.** However,parentheses can be used to force the order of evaluation of overloaded operators in an expression.
- **The associativity of an operator cannot be changed by overloading.** If an operator normally associates from left to right,then so do all of its overloaded versions.
- **You cannot change the "arity" of an operator** (that is, the number of operands an operator takes). Overloaded unary operators remain unary operators;overloaded binary operators remain binary operators.
- **You cannot create new operators; only existing operators can be overloaded**.

Rules and Restrictions on Operator Overloading

- The meaning of how an operator works on values of fundamental types cannot be changed by operator overloading.
- For example, you cannot make the + operator subtract two ints. **Operator overloading works only with objects of user-defined types or with a mixture of an object of a user-defined type and an object of a fundamental type.**
- **When overloading $(), [], ->$ or any of the assignment operators,the operator overloading function must be declared as a class member.** For all other overloadable operators,the operator overloading functions can be member functions or non-member functions.

Introduction

A binary operator can be overloaded as

- A **non-static member function with one parameter**
- A **non-member function with two parameters** (one of those parameters must be either a class object or a reference to a class object).

A non-member operator function is often declared as friend of a class for performance reasons.

Binary Overloaded Operators as Member Functions

- Consider using $<$ to compare two objects of a String class that you define.
- When overloading binary operator $<$ as a **non-static member function** of a String class, if y and z are String-class objects.
- Then y $<$ z is treated as if y.operator$<$(z) had been written, invoking the operator $<$ member function with one argument declared below:

```
class String
{
public:
   bool operator<( const String & ) const;
   ...
}; // end class String
```

- Overloaded operator functions for binary operators can be member functions only when the left operand is an object of the class in which the function is a member.

Binary Overloaded Operators as Member Functions

```
13  class Box {
14
15  public:
16      Box(double l=0,double b=0, double h=0){
17          length=l;
18          breadth=b;
19          height=h;
20      }
21      double getVolume(void) {
22          volume=length * breadth * height;
23          return volume;
24      }
25      // Overload + operator to add two Box objects.
26      Box operator+(const Box& b) {
27          Box box;
28          box.length = this->length + b.length;
29          box.breadth = this->breadth + b.breadth;
30          box.height = this->height + b.height;
31          return box;
32      }
33      // Overload < operator to compare two Box objects in terms of Volume.
34      bool operator<(const Box& b){
35          return this->volume > b.volume;
36      }
37
38  private:
39      double length;      // Length of a box
40      double breadth;     // Breadth of a box
41      double height;      // Height of a box
42      double volume;
43  };
```

Binary Overloaded Operators as Member Functions

```
46  main()
47  {
48      Box Box1(6,7,5);                // Declare Box1 of type Box
49      Box Box2(12,13,10);             // Declare Box2 of type Box
50      Box Box3;                       // Declare Box3 of type Box
51
52      cout << "Volume of Box1 : " << Box1.getVolume() <<endl;
53      cout << "Volume of Box2 : " << Box2.getVolume() <<endl;
54      cout << "Volume of Box3 : " << Box3.getVolume() <<endl;
55
56      // Add two object as follows:
57      Box3 = Box1 + Box2;
58      cout << "Volume of Box3 : " << Box3.getVolume() <<endl;
59
60      if (Box1<Box2)
61          cout << "Volume of Box1 is greater than that of Box2" << endl;
62      else
63          cout << "Volume of Box2 is greater than that of Box1" << endl;
64  }
```

Binary Overloaded Operators as Non-Member Functions

- As a non-member function, binary operator $<$ must take two arguments.
- One of which must be an object (or a reference to an object) of the class.
- If y and z are String-class objects or references to String-class objects, then y<z is treated as if the call operator<(y,z) had been written in the program, invoking function operator $<$ which is declared as follows:

```
bool operator<( const String &, const String & );
```

Binary Overloaded Operators as Non-Member Functions

```
13  class Box {
14
15      friend Box operator+(const Box& b1,const Box& b2);
16      friend bool operator<(const Box& b1,const Box& b2);
17
18  public:
19      Box(double l=0,double b=0, double h=0){
20          length=l;
21          breadth=b;
22          height=h;
23      }
24      double getVolume(void) {
25          volume=length * breadth * height;
26          return volume;
27      }
28
29  private:
30      double length;      // Length of a box
31      double breadth;     // Breadth of a box
32      double height;      // Height of a box
33      double volume;
34  };
```

Binary Overloaded Operators as Non-Member Functions

```
37  // Overload + operator to add two Box objects.
38  Box operator+(const Box& b1,const Box& b2) {
39      Box box;
40      box.length = b1.length + b2.length;
41      box.breadth = b1.breadth + b2.breadth;
42      box.height = b1.height + b2.height;
43      return box;
44  }
45
46  // Overload < operator to compare two Box objects in terms of Volume.
47  bool operator<(const Box& b1,const Box& b2){
48      return b1.volume > b2.volume;
49  }
50
51  // Main function for the program
52  main()
53  {
54      Box Box1(6,7,5);                // Declare Box1 of type Box
55      Box Box2(12,13,10);             // Declare Box2 of type Box
56      Box Box3;                       // Declare Box3 of type Box
57
58      cout << "Volume of Box1 : " << Box1.getVolume() <<endl;
59      cout << "Volume of Box2 : " << Box2.getVolume() <<endl;
60      cout << "Volume of Box3 : " << Box3.getVolume() <<endl;
61
62      // Add two object as follows:
63      Box3 = Box1 + Box2;
64      cout << "Volume of Box3 : " << Box3.getVolume() <<endl;
65
66      if (Box1<Box2)
67          cout << "Volume of Box1 is greater than that of Box2" << endl;
68      else
69          cout << "Volume of Box2 is greater than that of Box1" << endl;
70  }
```

Introduction

- You can input and output fundamental-type data using the stream extraction operator >> and the stream insertion operator <<.
- The C++ class libraries overload these binary operators for each fundamental type.
- You can also overload these operators to perform input and output for your own types.

PhoneNumber Class Definition

```
 9  #ifndef PHONENUMBER_H
10  #define PHONENUMBER_H
11
12  #include <iostream>
13  #include <string>
14  using namespace std;
15
16  // Rectangle class definition
17  class PhoneNumber
18  {
19
20      friend ostream &operator<<(ostream &, const PhoneNumber &);
21      friend istream &operator>>(istream &, PhoneNumber &);
22
23  private:
24
25      string areaCode;     // 3-digit area code
26      string exchange;     // 3-digit exchange
27      string line;         // 4-digit line
28
29  };
30
31  #endif
```

- The class definition contains friend function prototypes for overloaded << and >>.
- Notice that the PhoneNumber Class does not include any member function or data member in public access specifier.
- It also includes private data members as in string type (lines 25-27).

PhoneNumber Class Definition

```
 8  #include <iomanip>
 9  #include "PhoneNumber.h" // include definition of class PhoneNumber
10  using namespace std;
11
12  // overloaded stream insertion operator;cannot be
13  // a member function if we would like to invoke it with
14  // cout << somePhoneNumber;
15  ostream &operator<<(ostream &output, const PhoneNumber &number)
16  {
17      output<< "(" << number.areaCode << ") "
18      << number.exchange<< "-" << number.line;
19      return output; // enables cout << a << b << c;
20  }
21
22  // overloaded stream extractionoperator; cannot be
23  // a member function if we would like to invoke it with
24  // cin >> somePhoneNumber;
25  istream &operator>>(istream &input,PhoneNumber &number)
26  {
27      input.ignore(); // skip (
28      input >> setw( 3 ) >> number.areaCode; // input area code
29      input.ignore( 2 ); // skip ) and space
30      input >> setw( 3 ) >> number.exchange; // input exchange
31      input.ignore(); // skip dash(-)
32      input >> setw( 4 ) >> number.line; // input line
33      return input; // enables cin >> a >> b >> c;
34  }
```

- The stream extraction operator function operator >> (lines 25-34) takes the istream reference input and the PhoneNumber reference number as arguments and returns an istream reference.
- Operator function operator >> inputs phone numbers of the form (555) 555-1212 into objects of class PhoneNumber.
- When the compiler sees the expression cin >> phone, the compiler generates the non-member function call **operator**>>(cin,phone);
- When this call executes, reference parameter input (line 25) becomes an alias for cin and reference parameter number becomes an alias for phone.

PhoneNumber Class Definition

```
 8  #include <iomanip>
 9  #include "PhoneNumber.h" // include definition of class PhoneNumber
10  using namespace std;
11
12  // overloaded stream insertion operator;cannot be
13  // a member function if we would like to invoke it with
14  // cout << somePhoneNumber;
15  ostream &operator<<(ostream &output, const PhoneNumber &number)
16  {
17      output<< "(" << number.areaCode << ") "
18      << number.exchange<< "-" << number.line;
19      return output; // enables cout << a << b << c;
20  }
21
22  // overloaded stream extractionoperator; cannot be
23  // a member function if we would like to invoke it with
24  // cin >> somePhoneNumber;
25  istream &operator>>(istream &input,PhoneNumber &number)
26  {
27      input.ignore(); // skip (
28      input >> setw( 3 ) >> number.areaCode; // input area code
29      input.ignore( 2 ); // skip ) and space
30      input >> setw( 3 ) >> number.exchange; // input exchange
31      input.ignore(); // skip dash(-)
32      input >> setw( 4 ) >> number.line; // input line
33      return input; // enables cin >> a >> b >> c;
34  }
```

- The operator function reads as strings the three parts of the telephone number into the areaCode (line 28), exchange (line 30) and line (line 32) members of the PhoneNumber object referenced by parameter number.
- Stream manipulator setw limits the number of characters read into each string.
- When used with cin and strings, setw restricts the number of characters read to the number of characters specified by its argument (i.e., setw(3) allows three characters to be read).

PhoneNumber Class Definition

```
 8  #include <iomanip>
 9  #include "PhoneNumber.h" // include definition of class PhoneNumber
10  using namespace std;
11
12  // overloaded stream insertion operator;cannot be
13  // a member function if we would like to invoke it with
14  // cout << somePhoneNumber;
15  ostream &operator<<(ostream &output, const PhoneNumber &number)
16  {
17      output<< "(" << number.areaCode << ") "
18      << number.exchange<< "-" << number.line;
19      return output; // enables cout << a << b << c;
20  }
21
22  // overloaded stream extractionoperator; cannot be
23  // a member function if we would like to invoke it with
24  // cin >> somePhoneNumber;
25  istream &operator>>(istream &input,PhoneNumber &number)
26  {
27      input.ignore(); // skip (
28      input >> setw( 3 ) >> number.areaCode; // input area code
29      input.ignore( 2 ); // skip ) and space
30      input >> setw( 3 ) >> number.exchange; // input exchange
31      input.ignore(); // skip dash(-)
32      input >> setw( 4 ) >> number.line; // input line
33      return input; // enables cin >> a >> b >> c;
34  }
```

- The parentheses, space and dash characters are skipped by calling istream member function ignore (lines 27, 29 and 31), which discards the specified number of characters in the input stream(one character by default).
- Function operator >> returns istream reference input (i.e., cin). This enables input operations on PhoneNumber objects to be cascaded with input operations on other PhoneNumber objects or other data types.
- For example, a program can input two PhoneNumber objects in one statement as follows cin >> phone1 >> phone2;
- First, the expression cin >> phone1 executes by making the non-member function call **operator**>>(cin,phone1);

PhoneNumber Class Definition

```
 8  #include <iomanip>
 9  #include "PhoneNumber.h" // include definition of class PhoneNumber
10  using namespace std;
11
12  // overloaded stream insertion operator;cannot be
13  // a member function if we would like to invoke it with
14  // cout << somePhoneNumber;
15  ostream &operator<<(ostream &output, const PhoneNumber &number)
16  {
17      output<< "(" << number.areaCode << ") "
18      << number.exchange<< "-" << number.line;
19      return output; // enables cout << a << b << c;
20  }
21
22  // overloaded stream extractionoperator; cannot be
23  // a member function if we would like to invoke it with
24  // cin >> somePhoneNumber;
25  istream &operator>>(istream &input,PhoneNumber &number)
26  {
27      input.ignore(); // skip (
28      input >> setw( 3 ) >> number.areaCode; // input area code
29      input.ignore( 2 ); // skip ) and space
30      input >> setw( 3 ) >> number.exchange; // input exchange
31      input.ignore(); // skip dash(-)
32      input >> setw( 4 ) >> number.line; // input line
33      return input; // enables cin >> a >> b >> c;
34  }
```

- For example, a program can input two PhoneNumber objects in one statement as follows
  cin >> phone1 >> phone2;
- First, the expression cin >> phone1 executes by making the non-member function call
  operator>>(cin,phone1);
- This call then returns a reference to cin as the value of cin >> phone1, so the remaining portion of the expression is interpreted simply as cin >> phone2.
- This executes by making the non-member function call **operator**>>(cin,phone2);

PhoneNumber Class Definition

```
 8  #include <iomanip>
 9  #include "PhoneNumber.h" // include definition of class PhoneNumber
10  using namespace std;
11
12  // overloaded stream insertion operator;cannot be
13  // a member function if we would like to invoke it with
14  // cout << somePhoneNumber;
15  ostream &operator<<(ostream &output, const PhoneNumber &number)
16  {
17      output<< "(" << number.areaCode << ") "
18      << number.exchange<< "-" << number.line;
19      return output; // enables cout << a << b << c;
20  }
21
22  // overloaded stream extractionoperator; cannot be
23  // a member function if we would like to invoke it with
24  // cin >> somePhoneNumber;
25  istream &operator>>(istream &input,PhoneNumber &number)
26  {
27      input.ignore(); // skip (
28      input >> setw( 3 ) >> number.areaCode; // input area code
29      input.ignore( 2 ); // skip ) and space
30      input >> setw( 3 ) >> number.exchange; // input exchange
31      input.ignore(); // skip dash(-)
32      input >> setw( 4 ) >> number.line; // input line
33      return input; // enables cin >> a >> b >> c;
34  }
```

- The stream insertion operator function (lines 15-20) takes an ostream reference (output) and a const PhoneNumber reference (number) as arguments and returns an ostream reference.
- Function operator << displays objects of type PhoneNumber. When the compiler sees the expression cout << phone, the compiler generates the non-member function call **operator**<<(cout,phone);
- Function operator << displays the parts of the telephone numberas strings, because they're stored as string objects.

Overloaded Operators as Non-Member friend Functions

- **The functions operator>> and operator<< are declared in PhoneNumber as non-member, friend functions.**
- **They're non-member functions because the object of class PhoneNumber must be the operator's right operand.**
- If these were to be PhoneNumber member functions, the following awkward statements would have to be used to output and input:

```
phone << cout;
phone >> cin;
```

- Such statements would be confusing to most C++ programmers, who are familiar with cout and cin appearing as the left operands of << and >>, respectively.

Overloaded Operators as Non-Member friend Functions

- Overloaded operator functions for binary operators can be member functions only when the left operand is an object of the class in which the function is a member.
- Overloaded input and output operators are declared as friends if they need to access non-public class members directly for performance reasons or because the class may not offer appropriate get functions.
- Also,the PhoneNumber reference in function operator<<'s parameter list (line 15) is const, because the PhoneNumber will simply be output.
- The PhoneNumber reference in function operator>>'s parameter list (line 25) is non-const, because the PhoneNumber object must be modified to store the input telephone number in the object.

Introduction

A unary operator for a class can be overloaded as

- A **non-static member function no arguments**
- A **non-member function with one argument that must be an object** (or a reference to an object).

Member functions that implement overloaded operators must be non-static so that they can access the non-static data in each object of the class.

Unary Overloaded Operators as Member Functions

- Consider overloading unary operator ! to test whether an object of your own String class is empty.
- Such a function would return a bool result.
- When a unary operator such as ! is overloaded as a member function with no arguments and the compiler sees the expression !s (in which s is an object of class String), the compiler generates the function call s.operator!().
- The operand s is the String object for which the String class member function operator! is being invoked.
- The function is declared as follows:

```
class String
{
public:
   bool operator!() const;
   ...
}; // end class String
```

Unary Overloaded Operators as Member Functions

```
13  class Point
14  {
15  private:
16      double m_x, m_y, m_z;
17
18  public:
19      Point(double x=0.0, double y=0.0, double z=0.0):
20          m_x(x), m_y(y), m_z(z)
21      {
22      }
23
24      // Convert a Point into its negative equivalent
25      Point operator- () const;
26
27      // Return true if the point is set at the origin
28      bool operator! () const;
29
30      double getX() { return m_x; }
31      double getY() { return m_y; }
32      double getZ() { return m_z; }
33  };
34
35  // Convert a Point into its negative equivalent
36  Point Point::operator- () const
37  {
38      return Point(-m_x, -m_y, -m_z);
39  }
40
41  // Return true if the point is set at the origin, false otherwise
42  bool Point::operator! () const
43  {
44      return (m_x == 0.0 && m_y == 0.0 && m_z == 0.0);
45  }
```

Unary Overloaded Operators as Member Functions

```
47  main()
48  {
49      Point point; // use default constructor to set to (0.0, 0.0, 0.0)
50
51      if (!point)
52          cout << "point is set at the origin." << endl;
53      else
54          cout << "point is not set at the origin." << endl;
55
56     Point point2(-3,4,-5);
57
58     point2=-point2;
59
60      cout << "point2( " << point2.getX() << " , " << point2.getY()
61         << " , " << point2.getZ() << " )" << endl;
62
63  }
```

Unary Overloaded Operators as Non-Member Functions

- A unary operator such as ! may be overloaded as a non-member function with one parameter in two different ways:
- Either with a parameter that's an object(this requires a copy of the object, so the side effects of the function are not applied to the original object),
- Or with a parameter that's a reference to an object (no copy of the original object is made, so all side effects of this function are applied to the original object).
- If s is a String class object(or a reference to a String class object),then !s is treated as if the call operator!(s) had been written, invoking the non-member operator! function that's declared as follows:

```
bool operator!( const String & );
```

Unary Overloaded Operators as Non-Member Functions

```
13  class Point
14  {
15      // Convert a Point into its negative equivalent
16      friend Point operator- (Point &);
17
18      // Return true if the point is set at the origin
19      friend bool operator! (Point &);
20
21  private:
22      double m_x, m_y, m_z;
23
24  public:
25      Point(double x=0.0, double y=0.0, double z=0.0):
26          m_x(x), m_y(y), m_z(z)
27      {
28      }
29
30      double getX() { return m_x; }
31      double getY() { return m_y; }
32      double getZ() { return m_z; }
33  };
34
35  // Convert a Point into its negative equivalent
36  Point operator- (Point &p)
37  {
38      p.m_x=-p.m_x;
39      p.m_y=-p.m_y;
40      p.m_z=-p.m_z;
41      return p;
42  }
43
44  // Return true if the point is set at the origin, false otherwise
45  bool operator! (Point &p)
46  {
47      return (p.m_x == 0.0 && p.m_y == 0.0 && p.m_z == 0.0);
48  }
```

Unary Overloaded Operators as Non-Member Functions

```
47  main()
48  {
49      Point point; // use default constructor to set to (0.0, 0.0, 0.0)
50
51      if (!point)
52          cout << "point is set at the origin." << endl;
53      else
54          cout << "point is not set at the origin." << endl;
55
56     Point point2(-3,4,-5);
57
58     point2=-point2;
59
60      cout << "point2( " << point2.getX() << " , " << point2.getY()
61         << " , " << point2.getZ() << " )" << endl;
62
63  }
```

Introduction

- The prefix and postfix versions of the increment and decrement operators can all be overloaded.
- To overload the prefix and postfix increment operators, each overloaded operator function must have a distinct signature,so that the compiler will be able to determine which version of ++ is intended.
- The prefix versions are overloaded exactly as any other prefix unary operator would be.

Overloading the Prefix Increment Operator

- Suppose, that we want to add 1 to the day in Date object d1. When the compiler sees the pre incrementing expression ++d1, the compiler generates the **member-function** call
  d1.**operator**++()
- The prototype for this operator member function would be
  Date &**operator**++()
- If the prefix increment operator is implemented as a **non-member function**,then, when the compiler sees the expression ++d1,the compiler generates the function call
  operator++(d1)
- The prototype for this non-member operator function would be declared as
  Date &**operator**++(Date &)

Overloading the Prefix Increment Operator

- Overloading the postfix increment operator presents a challenge, because the compiler must be able to distinguish between the signatures of the overloaded prefix and postfix increment operator functions.
- The convention that has been adopted is that, when the compiler sees the postincrementing expression d1++,it generates the **member-function** call
  d1.**operator**++(0)
- The prototype for this operator member function would be
  Date **operator**++(**int**)
- The argument 0 is strictly a "dummy value" that enables the compiler to distinguish between the prefix and postfix increment operator functions.
- The same syntax is used to differentiate between the prefix and postfix decrement operator functions.

Overloading the Prefix Increment Operator

- If the postfix increment is implemented as a **non-member function**, then, when the compiler sees the expression d1++, the compiler generates the function call
  operator++(d1,0)
- The prototype for this non-member operator function would be declared as
  Date **operator**++(Date & ,**int**)
- Note that the postfix increment operator returns Date objects by value, whereas the prefix increment operator returns Date objects by reference, the postfix increment operator typically returns a temporary object that contains the original value of the object before the increment occurred.
- C++ treats such objects as rvalues, which cannot be used on the left side of an assignment. The prefix increment operator returns the actual incremented object with its new value.Such an object can be used as an lvalue in a continuing expression.

Overloading the Unary Prefix and Postfix ++ and −− Operators

```
13  class ComplexNumber
14  {
15      int real;
16      int imaginary;
17  public:
18
19      ComplexNumber(int r=0, int i=0) :
20              real(r), imaginary(i)
21      {}
22
23      // Prefix Decrement Operator
24      ComplexNumber operator--();
25
26      //Postfix Decrement Operator
27      ComplexNumber operator--(int);
28
29      void print (void) const
30      {
31          cout << "( " << real << " , "  << imaginary << " )" << endl;
32      }
33  };
34
```

Overloading the Unary Prefix and Postfix ++ and −− Operators

```
35  /*
36   *  Prefix Decrement Operator
37   *  It first decrements the Current Object and then returns its copy.
38   */
39  ComplexNumber ComplexNumber::operator--()
40  {
41      --real;
42      --imaginary;
43      return *this;
44  }
45
46  /*
47   *  Postfix Decrement Operator
48   *  It first creates a temp copy of current object. Then decrements
49   *  the current object. In the end returns the temp copy created initially.
50   */
51  ComplexNumber ComplexNumber::operator--(int)
52  {
53      // Temp copy of the current object
54      ComplexNumber tempObj(this->real, this->imaginary);
55      real--;
56      imaginary--;
57
58      // Return the temp copy that stores the data before updation
59      return tempObj;
60  }
```

Overloading the Unary Prefix and Postfix ++ and −− Operators

```
62  int main()
63  {
64      // Create a complex number 2 + i3
65      ComplexNumber c1(2, 3);
66      c1.print();
67
68      cout << "Testing Postfix Decrement" << endl;
69      ComplexNumber c2 = c1--; // Postfix Decrement
70
71      c2.print();
72      c1.print();
73
74      cout << "Testing Prefix Decrement" << endl;
75
76      ComplexNumber c3 = --c1; // Prefix Decrement
77
78      c3.print();
79      c1.print();
80
81      return 0;
82  }
```

Date Class Definition

```
15  // Date class definition
16  class Date
17  {
18      /// Friend Overloades << operator
19      friend ostream &operator<<( ostream &, const Date &);
20
21  public:
22
23      /// Default Constructor
24      Date(int m=1, int d=1, int y=1900);
25      /// Set month, day and year
26      void setDate(int,int,int);
27      /// Prefix increment operator
28      Date &operator++();
29      /// Postfix increment operator
30      Date operator++( int );
31      /// Add days, modify object
32      const Date&operator+=( int );
33      /// is date in a leap year?
34      static bool leapYear( int );
35      /// is date at the end of month?
36      bool endOfMonth( int ) const;
37
38  private:
39
40      int month;
41      int day;
42      int year;
43
44      /// Array of days per month
45      static const int days[];
46      /// Utility function for incrementing date
47      void helpIncrement();
48
49  };
```

- The Date header specifies that Date's public interface includes an overloaded stream insertion operator (line 19), a default constructor (line 24), an overloaded prefix increment operator (line 28), an overloaded postfix increment operator (line 30), an overloaded += addition assignment operator (line 32), a function to test for leap years (line 34) and a function to determine whether a day is the last day of the month (line 36)
- Also, private data members month, day, and year, a **static const int** array to store days of months for each month, and a utility function for incrementing date.

Date Class Implementation

```
17  /// Default Constructor
18  Date::Date(int m, int d, int y)
19  {
20      setDate(m,d,y);
21  }
22
23  /// Set month,day and year
24  void Date::setDate( int mm, int dd, int yy )
25  {
26      if (mm>=1 && mm<= 12)
27          month =mm;
28      else
29          throw invalid_argument( "Month must be 1-12" );
30
31      if (yy>=1900 && yy<=2100)
32          year =yy;
33      else
34          throw invalid_argument( "Year must be >=1900 and <= 2100" );
35
36      // test for a leap year
37      if (( month == 2 && leapYear( year )&&dd>= 1 && dd <= 29 )||
38          (dd>= 1 && dd <= days[ month ]) )
39              day= dd;
40      else
41          throw invalid_argument("Day is out of range for current month and year" );
42  }
43
44  /// Overloaded prefix increment operator
45  Date &Date::operator++()
46  {
47      helpIncrement(); // increment date
48      return *this; // reference returntocreate an lvalue
49  }
```

- The Date constructor (lines 18-21) calls setDate (lines 24- 42) to validate the month,day and year specified. Invalid values for the month, day or year result in invalid_argument exceptions.
- Overloading the prefix increment operator is straightforward. The prefix increment operator (lines 45-49) calls utility function helpIncrement to increment the date.
- The overloaded prefix increment operator returns a reference to the current Date object. This occurs because the current object, *this, is returned as a Date &.
- This enables a preincremented Date object to be used as an lvalue, which is how the built-in prefix increment operator works for fundamental types.

Date Class Implementation

```
51  /// Function to help increment the date
52  void Date::helpIncrement()
53  {
54      // day is not end of month
55      if (!endOfMonth( day ))
56          ++day; // increment day
57      else if (month < 12 ){ // day is end of month and month <12
58          ++month; // increment month
59          day = 1; // first day of new month
60      }
61      else{ // last day of year
62          ++year; // increment year
63          month = 1; // first month of new year
64          day = 1; // first day of new month
65      }
66  }
67
68  /// Overloaded postfix increment operator; note that the
69  /// dummy integer parameter does not have a parameter name
70  Date Date::operator++( int )
71  {
72      Date temp =*this; // hold current state of object
73      helpIncrement();
74
75      // return unincremented,saved, temporary object
76      return temp; // value return; nota reference return
77  }
78
79  /// Add specified number of days to date
80  const Date &Date::operator+=( int additionalDays )
81  {
82      for ( int i= 0;i <additionalDays; ++i )
83          helpIncrement();
84
85      return *this; // enables cascading
86  }
```

- Overloading the postfix increment operator (lines 70-77) is trickier. To emulate the effect of the postincrement, we must return an unincremented copy of the Date object.
- For example, if int variable x has the value 7,the statement

  cout << x++ << endl;
- outputs the original value of variable x. So we'd like our postfix increment operator to operate the same way on a Date object.
- On entry to operator++, we save the current object (*this) in temp (line 72). Next, we call helpIncrement to increment the current Date object. Then,line 76 returns the unincremented copy of the object previously stored in temp.

Date Class Implementation

```
51  /// Function to help increment the date
52  void Date::helpIncrement()
53  {
54      // day is not end of month
55      if (!endOfMonth( day ))
56          ++day; // increment day
57      else if (month < 12 ){ // day is end of month and month <12
58          ++month; // increment month
59          day = 1; // first day of new month
60      }
61      else{ // last day of year
62          ++year; // increment year
63          month = 1; // first month of new year
64          day = 1; // first day of new month
65      }
66  }
67
68  /// Overloaded postfix increment operator; note that the
69  /// dummy integer parameter does not have a parameter name
70  Date Date::operator++( int )
71  {
72      Date temp =*this; // hold current state of object
73      helpIncrement();
74
75      // return unincremented,saved, temporary object
76      return temp; // value return; nota reference return
77  }
78
79  /// Add specified number of days to date
80  const Date &Date::operator+=( int additionalDays )
81  {
82      for ( int i= 0;i <additionalDays; ++i )
83          helpIncrement();
84
85      return *this; // enables cascading
86  }
```

- This function cannot return a reference to the local Date object temp,because a local variable is destroyed when the function in which it's declared exits.
- Thus, declaring the return type to this function as Date & would return a reference to an object that no longer exists.
- Returning a reference (or a pointer) to a local variable is a common error for which most compilers will issue a warning.

Date Class Implementation

```
51  /// Function to help increment the date
52  void Date::helpIncrement()
53  {
54      // day is not end of month
55      if (!endOfMonth( day ))
56          ++day; // increment day
57      else if (month < 12 ){ // day is end of month and month <12
58          ++month; // increment month
59          day = 1; // first day of new month
60      }
61      else{ // last day of year
62          ++year; // increment year
63          month = 1; // first month of new year
64          day = 1; // first day of new month
65      }
66  }
67
68  /// Overloaded postfix increment operator; note that the
69  /// dummy integer parameter does not have a parameter name
70  Date Date::operator++( int )
71  {
72      Date temp =*this; // hold current state of object
73      helpIncrement();
74
75      // return unincremented,saved, temporary object
76      return temp; // value return; nota reference return
77  }
78
79  /// Add specified number of days to date
80  const Date &Date::operator+=( int additionalDays )
81  {
82      for ( int i= 0;i <additionalDays; ++i )
83          helpIncrement();
84
85      return *this; // enables cascading
86  }
```

- helpIncrement function (lines 52-66) deals with "wraparounds" or "carries" that occur when we increment the last day of the month.
- These carries require incrementing the month. If the month is already 12, then the year must also be incremented and the month must be set to 1. Function helpIncrement uses function endOfMonth to increment the day correctly.
- The overloaded addition assignment operator (lines 80-86) add specified number of days to date.

Date Class Implementation

```
88  /// If the year is a leap year, return true; otherwise, return false
89  bool Date::leapYear( int testYear )
90  {
91      if (testYear % 400 == 0 || (testYear % 100 != 0 && testYear % 4 == 0 ))
92          return true; // a leap year
93      else
94          return false; // not a leap year
95  }
96
97  // Determine whether the day is the last day of the month
98  bool Date::endOfMonth( int testDay ) const
99  {
100     if (month== 2 && leapYear(year) )
101         return testDay == 29; // last day of Feb. in leap year
102     else
103         return testDay == days[ month ];
104 }
105
106 /// Overloaded output operator
107 ostream &operator<<( ostream &output, const Date &d )
108 {
109     static string monthName[ 13 ]= { "", "January", "February",
110         "March", "April", "May", "June", "July", "August",
111         "September", "October", "November", "December" };
112
113     output<<monthName[ d.month]<< " " << d.day<< ", " << d.year;
114     return output; // enables cascading
115 }
```

Driver Program for Date Class

```
13 int main()
14 {
15     Date d1(12,27,2010); // December 27, 2010
16     Date d2; // defaults to January 1, 1900
17
18     cout << "d1 is " << d1 << "\nd2 is " << d2;
19     cout << "\n\nd1+=7 is " << (d1 += 7);
20
21     d2.setDate( 2, 28, 2008 );
22     cout << "\n\nd2 is " << d2;
23
24     cout << "\n++d2 is " << ++d2 << "(leap year allows 29th)";
25
26     Date d3( 7, 13, 2010 );
27
28     cout << "\n\nTesting the prefix increment operator:\n" << "d3 is " << d3 << endl;
29     cout << "++d3 is " << ++d3 << endl;
30     cout << "d3 is" << d3;
31
32     cout << "\n\nTesting the postfix increment operator:\n" << "d3 is " << d3 << endl;
33     cout << "d3++ is " << d3++ << endl;
34     cout << "d3 is " << d3 << endl;
35
36 }
```

- Function main creates two Date objects(lines 15-16)
- d1 is initialized to December 27,2010 and d2 is initialized by default to January 1,1900.
- Line 18 of main outputs each of the Date objects,using the overloaded stream insertion operator.
- Line 19 of main uses the overloaded operator += to add seven days to d1.
- Line 21 uses function setDate to set d2 to February 28, 2008, which is a leap year.
- Then, line 24 preincrements d2 to show that the date increments properly to February 29.
- Next, line 26 creates a Date object, d3, which is initialized with the date July 13, 2010.

Driver Program for Date Class

```
13  int main()
14  {
15      Date d1(12,27,2010); // December 27, 2010
16      Date d2; // defaults to January 1, 1900
17
18      cout << "d1 is " << d1 << "\nd2 is " << d2;
19      cout << "\n\nd1+=7 is " << (d1 += 7);
20
21      d2.setDate( 2, 28, 2008 );
22      cout << "\n\nd2 is " << d2;
23
24      cout << "\n++d2 is " << ++d2 << "(leap year allows 29th)";
25
26      Date d3( 7, 13, 2010 );
27
28      cout << "\n\nTesting the prefix increment operator:\n" << "d3 is " << d3 << endl;
29      cout << "++d3 is " << ++d3 << endl;
30      cout << "d3 is" << d3;
31
32      cout << "\n\nTesting the postfix increment operator:\n" << "d3 is " << d3 << endl;
33      cout << "d3++ is " << d3++ << endl;
34      cout << "d3 is " << d3 << endl;
35
36  }
```

- Then, line 29 increments d3 by 1 with the overloaded prefix increment operator. Lines 28-30 output d3 before and after the preincrement operation to confirm that it worked correctly.
- Finally, line 33 increments d3 with the overloaded postfix increment operator. Lines 32-34 output d3 before and after the postincrement operation to confirm that it worked correctly.